AI Project Proposal: Auto File Organizer

Team Members: Aaron Baclor, Tristan Noval, Jarelle Ricaforte, Gabrielle Sacramento
Course: CS 180 – AI for Everyday Life
Date: February 22, 2026

---

1. Project Overview

Application Name: OrgAIzer
Theme/Category: Productivity & Personal Automation
Core AI Technology: Machine Learning

2. Problem Statement

Everyday Problem

In everyday computer use, the Downloads folder frequently becomes a high-volume "catch-all" location for documents, installers, images, and miscellaneous files. Because users often postpone manual organization, important files become difficult to retrieve later, leading to wasted time searching, duplicate downloads, and reduced personal productivity.

Target Users

The primary users are students and knowledge workers who regularly download academic, administrative, and work-related files (e.g., lecture materials, assignments, forms, receipts, and project documents). These users experience repeated friction from unmanaged file accumulation and benefit from faster, more consistent file organization and retrieval.

Why AI Is the Right Tool

Traditional rule-based file organization tools generally depend on fixed heuristics, such as sorting by file extension, download source, or manually defined keyword rules. While these methods are simple to implement, they often fail to reflect how users actually categorize files in real settings. File organization is frequently semantic and context-driven. The same file type may belong to different folders depending on its topic, source, or purpose.

Thus, a supervised machine learning approach is appropriate because it can learn a user's implicit file-organization behavior directly from observed actions rather than relying on fixed, manually maintained rules. Moreover, machine learning supports an adaptive feedback loop: when the system's suggestion is accepted or corrected, that interaction can be logged and used to periodically retrain the model, allowing performance to improve over time and remain robust as the user's habits and folder structure evolve, which static heuristic approaches typically cannot handle effectively.

3. Proposed Solution

It will be a desktop application that watches the user's Downloads folder and learns, purely from observing the user's own file-moving behavior, where each type of file should go. It combines a bootstrapped general classifier with a personal behavioral learning layer that continuously adapts to the individual user.

Expected User Workflow

1. A file lands in the Downloads folder.
2. The app extracts features: filename tokens, file extension, and content keywords (via OCR or text extraction).
3. The ML model predicts the destination folder and returns a confidence score.
4. High confidence (≥ 90%): file is moved automatically. Medium confidence (60–89%): a notification suggests a folder. Low confidence (< 60%): the user is prompted to choose.
5. The user's action (accept, override, or manual move) is logged as a new training sample and the model is incrementally retrained.

4. Technical Approach

4.1 Data Sourcing

The dataset will be generated directly from user behavior during a 1–2 week observation period. Instead of relying on an external dataset, the system collects personalized labeled data by monitoring how users organize their files.

A file system listener (e.g., Python's *watchdog* library) will monitor the Downloads folder for file creation and file movement events. When a user manually moves a file from Downloads to another folder, the system records:

- File name
- File extension
- File size
- Time stamp
- Destination folder

Each movement becomes one labeled training example, where the extracted file features serve as the input and the destination folder serves as the label.

The learning objective is to model the mapping:

$$\hat{y} = f(X)$$

Where:

- X represents the feature vector of a file
- $\hat{y}$ represents the predicted destination folder
- f is the trained classification model

All data will be stored locally to ensure privacy and security.

During the observation period, we expect to collect approximately 200–500 labeled file movements per user. The dataset will continue to grow as the system receives corrections over time.

4.2 Model Architecture

The system will use multinomial logistic regression as its primary machine learning model for classifying files into destination folders. The model takes as input features derived from each file, including filename text, file type, and basic metadata, and outputs a probability distribution over possible folders. The predicted probabilities are used both to select the most likely destination and to determine whether the system should automatically move the file or request user confirmation.

Logistic regression was chosen because it performs well on text-based classification tasks and remains effective even with relatively small datasets, which aligns with the expected scale of user-generated data. It also produces reliable probability estimates, which are necessary for confidence-based decision-making in the system. Additionally, the model is computationally efficient and can be retrained frequently as new user behavior is observed, making it suitable for a continuously adapting, personalized application.

4.3 Evaluation Metrics

The system will be evaluated using a combination of accuracy-based and user-centered metrics to capture both technical performance and practical usability. The primary metric will be Top-1 accuracy, which measures how often the model correctly predicts the destination folder. Top-3 accuracy will also be used to evaluate the quality of suggested folders when the system operates in recommendation mode.

To ensure reliability in automated actions, the accuracy of high-confidence predictions will be specifically monitored. Additionally, performance across different folders will be assessed using standard classification metrics to account for potential class imbalance. From a user perspective, the override rate and automation rate will be tracked to measure how often the system makes correct decisions without intervention and how frequently users need to correct it. Overall success will be defined by high prediction accuracy, low correction rates, and increasing automation over time.

5. Potential Challenges

Cold Start Problem

During the initial phase, the system will have limited labeled data, which may result in low prediction accuracy. Since the model relies entirely on user-generated training examples, early predictions may be incorrect or inconsistent. This may require a short observation period before enabling automatic file movement to maintain user trust.

Limited Data Size

Because the dataset is personalized per user, the total number of labeled examples may be relatively small (e.g., 200–500 samples). Small datasets can limit model generalization and increase the risk of overfitting, especially if certain folders have significantly more examples than others.

Class Imbalance

Some folders (e.g., “School” or “Work”) may receive many more files than others. This imbalance may cause the classifier to favor majority classes and reduce prediction accuracy for less frequently used folders.

Feature Ambiguity

Filenames may not always contain clear semantic signals. For example, generic names such as “document.pdf” or “image1.png” provide minimal useful information. Without strong textual cues, the model may struggle to confidently predict the correct destination.

Concept Drift

User organization preferences may change over time. For example, a user may create new folders or begin organizing files differently during a new semester or job. The model must be periodically retrained to adapt to these changes, otherwise performance may degrade.

Real-Time System Reliability
Monitoring file system events and automatically moving files introduces technical risks such as:

- Incorrect file movement
- Race conditions when files are still downloading
- Handling duplicate filenames
- Ensuring no file loss occurs

6. Project Timeline

- Data Collection & Cleaning (Due Date): April 5, 2026
- Model Training & Evaluation (Due Date): April 20, 2026
- UI Development & Integration (Due Date): April 29, 2026
- Final Report & Demo Preparation (Due Date): May 22, 2026